

2303A52416

assignment=15.1

Task 1 – Student Records API

Task:

Use AI to build a RESTful API for managing student records.

Instructions:

- Endpoints required:
  - o GET /students → List all students
  - o POST /students → Add a new student
  - o PUT /students/{id} → Update student details
  - o DELETE /students/{id} → Delete a student record
- Use an in-memory data structure (list or dictionary) to store records.
- Ensure API responses are in JSON format.

Expected Output:

- Working API with CRUD functionality for student records.

The image displays two screenshots of the OnlineGDB IDE, showing a FastAPI application for managing students. The interface includes a sidebar with navigation links and a main editor area with a code editor and a terminal.

**First Screenshot (Top):** The code defines a FastAPI application with the following endpoints:

- `GET /students`: Returns the list of students.
- `POST /students`: Adds a new student.
- `PUT /students/{id}`: Updates a student's information.

**Second Screenshot (Bottom):** The code is updated to include a `DELETE` endpoint:

- `DELETE /students/{id}`: Deletes a student.

## Explanation

This API is used to **manage student information** such as name and age. The data is stored in an **in-memory dictionary** called `students`.

## Key Concepts

- **REST API:** A web service that allows communication using HTTP methods.
- **CRUD Operations:**
  - **Create** → Add student
  - **Read** → View student data

- **Update** → Modify student details
- **Delete** → Remove student

## Endpoint Explanation

### 1 GET /students

- Retrieves all student records.
- Returns the **students** dictionary in JSON format.

## Task 2 – Library Book Management API

Task:

Develop a RESTful API to handle library books.

Instructions:

- Endpoints required:
  - GET /books → Retrieve all books
  - POST /books → Add a new book
  - GET /books/{id} → Get details of a specific book
  - PATCH /books/{id} → Update partial book details (e.g., availability)
  - DELETE /books/{id} → Remove a book
- Implement error handling for invalid requests.

Expected Output:

- Functional API with CRUD + partial updates.

The image displays two screenshots of the OnlineGDB IDE, showing Python code for a FastAPI application. The top screenshot shows the initial code, and the bottom screenshot shows the code after adding a delete endpoint.

**Top Screenshot Code:**

```
1 from fastapi import FastAPI
2
3 app = FastAPI()
4
5 books = {}
6
7 @app.get("/books")
8 def get_books():
9     return books
10
11 @app.post("/books")
12 def add_book(id: int, title: str, author: str):
13     books[id] = {"title": title, "author": author, "available": True}
14     return {"message": "Book added"}
15
16 @app.get("/books/{id}")
17 def get_book(id: int):
18     if id in books:
19         return books[id]
20     return {"error": "Book not found"}
21
22 @app.patch("/books/{id}")
23 def update_book(id: int, available: bool):
24     if id in books:
```

**Bottom Screenshot Code:**

```
11 @app.post("/books")
12 def add_book(id: int, title: str, author: str):
13     books[id] = {"title": title, "author": author, "available": True}
14     return {"message": "Book added"}
15
16 @app.get("/books/{id}")
17 def get_book(id: int):
18     if id in books:
19         return books[id]
20     return {"error": "Book not found"}
21
22 @app.patch("/books/{id}")
23 def update_book(id: int, available: bool):
24     if id in books:
25         books[id]["available"] = available
26         return {"message": "Book updated"}
27     return {"error": "Book not found"}
28
29 @app.delete("/books/{id}")
30 def delete_book(id: int):
31     if id in books:
32         del books[id]
33         return {"message": "Book deleted"}
34     return {"error": "Book not found"}
```

## Explanation

This API manages **library book records** such as title, author, and availability.

A dictionary called **books** stores book details.

### Task 3 – Employee Payroll API

Task:

Create an API for managing employees and their salaries.

Instructions:

- Endpoints required:

- o GET /employees → List all employees
- o POST /employees → Add a new employee with salary details
- o PUT /employees/{id}/salary → Update salary of an employee
- o DELETE /employees/{id} → Remove employee from system
- Use AI to:
  - o Suggest data model structure.
  - o Add comments/docstrings for all endpoints.

Expected Output:

- API supporting salary management with clear documentation.

```

1 from fastapi import FastAPI
2
3 app = FastAPI()
4
5 employees = {}
6
7 @app.get("/employees")
8 def get_employees():
9     return employees
10
11 @app.post("/employees")
12 def add_employee(id: int, name: str, salary: int):
13     employees[id] = {"name": name, "salary": salary}
14     return {"message": "Employee added"}
15
16 @app.put("/employees/{id}/salary")
17 def update_salary(id: int, salary: int):
18     if id in employees:
19         employees[id]["salary"] = salary
20         return {"message": "Salary updated"}
21     return {"error": "Employee not found"}
22
23 @app.delete("/employees/{id}")
24 def delete_employee(id: int):

```

```

25     if id in employees:
26         del employees[id]
27         return {"message": "Employee removed"}
28     return {"error": "Employee not found"}

```

# Explanation

This API manages **employee salary records**

Task 4 – Real-Time Application: Online Food Ordering API  
Scenario:

Design a simple API for an online food ordering system.

Requirements:

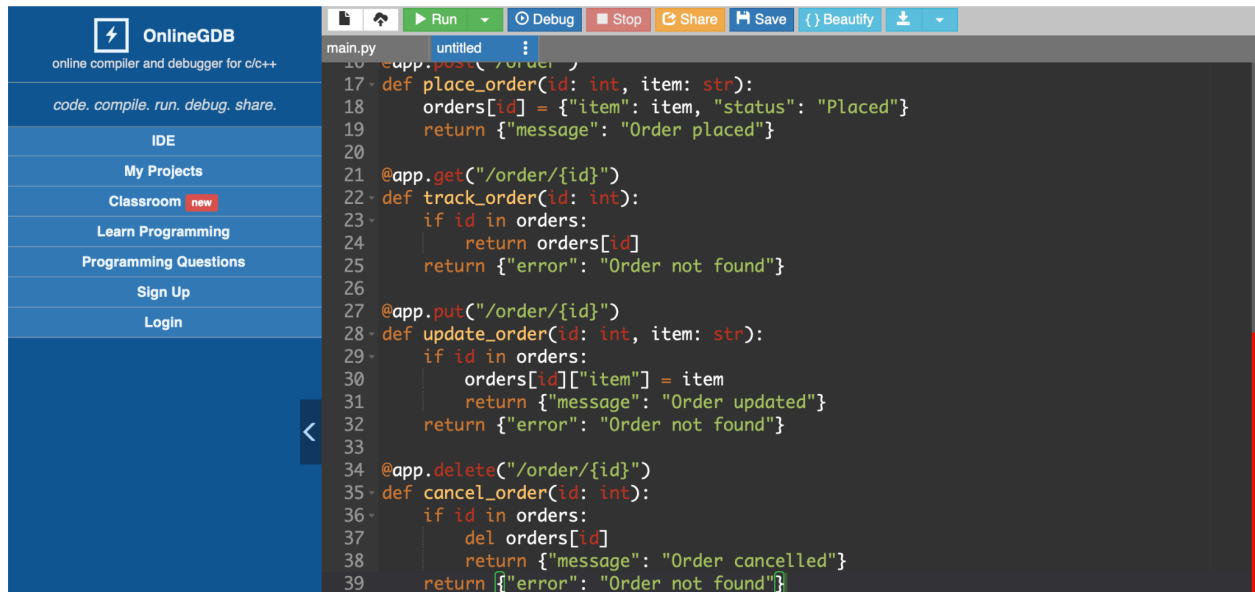
- Endpoints required:
  - o GET /menu → List available dishes
  - o POST /order → Place a new order
  - o GET /order/{id} → Track order status
  - o PUT /order/{id} → Update an existing order (e.g., change items)
  - o DELETE /order/{id} → Cancel an order
- AI should generate:
  - o REST API code
  - o Suggested improvements (like authentication, pagination)

Expected Output:

- Fully working API simulating a food ordering backend.
- ☐ Deliverables (For All Tasks)
1. AI-generated prompts for code and test case generation.
  2. At least 3 assert test cases for each task.
  3. AI-generated initial code and execution screenshots.
  4. Analysis of whether code passes all tests.
  5. Improved final version with inline comments and explanation.
  6. Compiled report (Word/PDF) with prompts, test cases,



```
1 from fastapi import FastAPI
2
3 app = FastAPI()
4
5 menu = {
6     1: {"dish": "Pizza", "price": 200},
7     2: {"dish": "Burger", "price": 120}
8 }
9
10 orders = {}
11
12 @app.get("/menu")
13 def get_menu():
14     return menu
15
16 @app.post("/order")
17 def place_order(id: int, item: str):
18     orders[id] = {"item": item, "status": "Placed"}
19     return {"message": "Order placed"}
20
21 @app.get("/order/{id}")
22 def track_order(id: int):
23     if id in orders:
24         return orders[id]
```



```
10 @app.post("/order")
11
17 def place_order(id: int, item: str):
18     orders[id] = {"item": item, "status": "Placed"}
19     return {"message": "Order placed"}
20
21 @app.get("/order/{id}")
22 def track_order(id: int):
23     if id in orders:
24         return orders[id]
25     return {"error": "Order not found"}
26
27 @app.put("/order/{id}")
28 def update_order(id: int, item: str):
29     if id in orders:
30         orders[id]["item"] = item
31         return {"message": "Order updated"}
32     return {"error": "Order not found"}
33
34 @app.delete("/order/{id}")
35 def cancel_order(id: int):
36     if id in orders:
37         del orders[id]
38         return {"message": "Order cancelled"}
39     return {"error": "Order not found"}
```

## Explanation

This API simulates a **simple food ordering system**.